Lab 3: Interpreting Methods

Goal: Extend your statement interpreter from Lab 2 to support procedure-oriented programming: an entry method, custom method declarations, method calls with overload resolution, array types, local variable type inference, and a set of built-in methods.

Deadline: 12:00:00 May 18, 2026 (*One week extension if necessary*)

Programming Language: Java Only.

Task 1: Port Your Lab 2 Implementation

1.1 What Changed in the Grammar

In Lab 3, the entry point is a **sequence of method declarations**:

```
compilationUnit : methodDeclaration* EOF
```

The program's execution begins at the `int main()` method. All other grammar rules for expressions, statements, blocks, and control flow remain compatible with Lab 2. The grammar has been extended with new rules for *method declarations, array types, `return`, and method call expressions*.

The project structure is identical to Lab 2:

```
lab3-methods/
├── pom.xml
├── src/main/java/cn/edu/nju/cs/
│   ├── Main.java                      # Entry point
│   ├── MiniJavaLexer.g4               # Updated lexer grammar
│   ├── MiniJavaParser.g4              # Updated parser grammar
│   ├── MiniJavaLexer.java             # Generated (do not modify)
│   ├── MiniJavaParser.java            # Generated (do not modify)
│   ├── MiniJavaParserBaseVisitor.java # Generated base visitor to extend
│   └── ...
└── testcase/
    ├── Array-1.mj
    ├── Method-1.mj
    └── ...
```

1.2 What You Need to Do

- Copy the `testcase` and `MiniJava*.g4` files from the Lab 3 starter project into your existing Lab 2 project. Do not forget to regenerate the ANTLR sources after updating the grammar files.
- Adapt your visitor methods to the new grammar. The top-level visitor entry point should now be `visitCompilationUnit`, which locates and calls `int main()`.

Aha! Start from Lab3, We **do not** dump all variables when exiting a Scope.

Task 1: Methods

1.1 Method Declaration

```
methodDeclaration
    : (typeType | VOID) identifier formalParameters
    methodBody = block
    ;
```

Methods are declared at the top level of the program. The scope of every method is the **entire compilation unit**.

Note 1: Forward references are allowed.

```
int main() {
    foo();
    return 0;
}
void foo() { ... }
```

1.2 `return` Statement

- For `void` methods: `return` is optional. If present, only empty return value is allowed.
- For **non-`void`** methods: `return expr;` is required, and a return value **must** be provided.

```
void foo1() { }
void foo2() { return; }

int foo3() {
    return 10;
}

int foo4() {                // illegal – missing return statement
    int res = 10;
}
```

Note 1: For invalid return statements (conflicting type or missing value), an error will be reported when the return statement was evaluated.

Note 2: For missing return statements in non-`void` methods, an error will be reported when the method finishes execution.

1.3 Implicit Type Conversion in Method Calls

Original	Target	Allowed
`char`	`int`	Yes
`null`	Array	Yes
`DECIMAL_LITERAL` (return statement)	`int` or `char`	yes
other	other	No

Note 1: The `DECIMAL_LITERAL` will always be treated as `int` when passed as an argument.

Note 2: The `DECIMAL_LITERAL` only can be treated as either a `char` or an `int` type.

```
int main() {
    foo(10);
    foo('a');
    bar(null);

    // foo2 returns char
    // which can not be implicitly converted to int when assignment
    int x = foo2(); // illegal, type error

    return 0;
}
void foo(int x) { ... }
void bar(int[] arr) { ... }

char foo2() { return 10; } // legal, implicit conversion from int to char
```

1.4 Method Overloading

Multiple methods with the same name but different parameter counts or types are allowed. When more than one method matches a call, **overload resolution** selects the best match:

1. The method name must match.
2. The number of parameters must equal the number of arguments.
3. Each parameter type must be compatible with the corresponding argument (same type or compatible via implicit conversion).
4. Among all compatible candidates, the one with the smallest **conversion-count** (number of implicit type conversions included) is selected.

Note 1: If multiple candidates have the same conversion-count, the call is ambiguous and results in an error.

Note 2: Currently, the conversion-count will only can be `0` or `1`.

```
int main() {
    foo(10);       // illegal - no single-argument foo
    foo(10,10,10); // illegal - no three-argument foo
    foo(10, 'b');  // matches foo@1, conversion-count = 0
    foo('a', 10);  // matches foo@2, conversion-count = 1
    foo('a', 'b'); // matches foo@1, conversion-count = 1
    foo(10, null); // matches foo@3, conversion-count = 1

    foo(null);     // illegal - ambiguous between foo@4 and foo@5
    return 0;
}
void foo(int x1, char x2);   // foo@1
void foo(int x1, int x2);    // foo@2
void foo(int x1, int[] x2);  // foo@3
void foo(int x1[]);          // foo@4
void foo(char x1[]);         // foo@5
```

Task 2: Entry Method

2.1 Program Structure

Every Lab 3 program should consist of one or more method declarations. The interpreter starts execution from the **entry method**:

Characteristics of the entry method:

- The name must be `main`.
- The return type must be `int`.
- It must have no parameters.

Note 1: if `void main()` and `int main()` both exist, the call to `main()` is ambiguous and results in an error when calling `main()`.

Note 2: if there is any other overloaded method named `main` (e.g. `void main(int x)`), it does not affect the entry method selection.

2.2 Process Exit Output

When `main` returns normally, the interpreter prints the following line and exits **with the return value as the exit code**:

```
Process exits with <return_value>.
```

When a runtime error occurs, the interpreter exits immediately with the corresponding error message and exit code (see Input and Output Format).

```
int main() {
    return 0;
}
```

Output:

```
Process exits with 0.
```

Task 3: Type Checking

Lab 3 requires strict type checking.

3.1 Primitive Type Rules

Please reference to the Implicit Type Conversion for implicit type conversion rules between primitive types.

3.2 `DECIMAL_LITERAL` Rules

A `DECIMAL_LITERAL` (e.g. `42`) can be treated as either `int` or `char` depending on context, **except** in the following cases where it must be treated as `int`:

1. Its value exceeds the range representable by `char` (-128 to 127).
2. During local variable type inference (`var`).
3. During method argument passing.
4. When operating with `string`.

3.3 `null` and Array Type

`null` has no intrinsic type but can be implicitly converted to any **Array**.

```
int[] arr = null;
int x = null;       // illegal, type error
```

Task 4: Array Types

Lab 3 introduces array types. The default value of an uninitialized array variable is `null`.

4.1 Array Initialization

Arrays can be initialized in several ways:

Note 1: The array must initialize elements with ordered from left to right, top to down.

Note 2: When initializing an array, the element types must be compatible with the array type. otherwise, a type error occurs when an element with conflicting type is assigned.

```
// One-dimensional arrays
int[] arr1;
int[] arr2 = null;
int[] arr3 = {1, 2, 3};
int[] arr4 = {1, 2, 3, };
int[] arr5 = {'a', 'b', 'c'};
int[] arr6 = new int[] {1, 2, 3};
int[] arr7 = new int[5];
int[] arr8 = arr7;
int[] arr9 = "123";                // illegal, type error
char[] cArr = {10, 20, 30};

// Multidimensional arrays
int[][] arr10 = {{1, 2}, {3, 4}};
int[][] arr11 = {null, null};
int[][] arr12 = new int[2][];
```

4.2 Array Assignment

Arrays of the same type can be assigned to each other. `null` is assignable to any array type.

```
int[] arr;
int[] temp = {1, 2, 3};
arr = temp;
arr = null;
arr = new int[] {1,2,3};
arr = new int[3];
arr = "123";             // illegal, type error
```

4.3 Array Comparison

Array comparison with `==` / `!=` checks **reference equality** — whether two variables point to the same object (consistent with Java). `null` is comparable with any array type.

```
int[] arr1 = null;
boolean b1 = arr1 == null;  // true
int[] arr2 = {1, 2, 3};
int[] arr3 = {1, 2, 3};
boolean b2 = arr2 == arr3;  // false (different objects)
boolean b3 = arr2 == 50;    // illegal, type error
```

4.4 Array Access

Elements are accessed via an index expression `arr[i]`. The index must be of type `int` (`char` is implicitly converted to `int`).

Legal index range: **[0, length)**.

Situation	Result
Valid index	Element value
Index out of range	Array out-of-bounds Error
Array is `null`	Null pointer Error
Non-integral index	Type Error

```
int[] arr = {1, 2, 3};
int x1 = arr[0];
int x2 = arr[3];     // illegal, array out-of-bounds
arr = null;
int x3 = arr[0];     // illegal, null pointer
int x4 = arr[true];  // illegal, type
```

Note: The interpreter should print error messages and exit immediately when an error occurs. Please refer to Task 7 for details of output format and exit codes.

Task 5: Type Inference

Lab 3 adds the `var` keyword for local variable type inference. The type is inferred from the initializer expression.

Note: A `DECIMAL_LITERAL` initializer is always inferred as `int`.

```
var x1 = 1; // int
var x2 = 'a'; // char
var x3 = "123"; // string
var x4 = true; // boolean
var x5 = new int[] {1,2,3}; // int[]
var x6 = null; // illegal, can not infer type for variable initialized to
'null'
```

Task 6: Built-in Methods

The following built-in methods are available in every program without any declaration.

Note: It is not necessary to consider the built-in functions was decleared by user-defined functions.

`print`

```
void print(T x)
```

Accepts one argument of any primitive type, array type, or `null`. Prints the argument to standard output **without** a trailing newline.

```
print(1);         // output: 1
print('a');       // output: a
print("Hello");   // output: Hello
print(null);      // output: null

int[] arr1 = {1, 2, 3};
print(arr1);      // output: [1, 2, 3]

int[][] arr2 = {{1,2},{3,4}};
print(arr2);      // output: [[1, 2], [3, 4]]

int[] arr3 = null;
print(arr3);      // output: null
```

Note: You do not need to support escape characters (e.g. `\n`, `\t`) in string arguments. Our test cases will not include them.

`println`

```
void println()
void println(T x)
```

With zero arguments: prints a newline. With one argument: behaves identically to `print`, but appends a newline at the end.

assert

```
void assert(boolean b)
```

Accepts exactly one `boolean` argument. `assert` is also a keyword in MiniJava, but it is only used as a built-in method call.

- If the condition is `true`: no-op.
- If the condition is `false`: assertion error — the interpreter exits with code **33** and prints `Process exits with 33.`.

```
assert(1 == 1); // ok
assert(1 == 0); // assertion error, exit 33
```

length

```
int length(T[] arr)
int length(string str)
```

Returns the number of elements (for arrays) or characters (for strings). If the argument is `null`, a null pointer error occurs.

```
length("123");          // 3
int[] arr = {1, 2, 3};
length(arr);            // 3
arr = null;
length(arr);            // null pointer error
```

to_char_array

```
char[] to_char_array(string str)
```

Converts a `string` to a `char[]` containing its characters.

```
char[] cArr = to_char_array("123"); // ['1', '2', '3']
```

to_string

```
string to_string(char[] arr)
```

Converts a `char[]` to a `string`. If the argument is `null`, a null pointer error occurs.

```
char[] cArr = {'1', '2', '3'};
string str = to_string(cArr); // "123"
```

atoi

```
int atoi(string str)
```

Parses a `string` and returns its integer value.

```
int x = atoi("10"); // 10
```

itoa

```
string itoa(int x)
```

Returns the decimal string representation of an integer. Note that `char` is implicitly converted to `int` when passed as an argument.

```
string s = itoa(10); // "10"
```

Task 7: Input and Output Format

7.1 Normal Output

All output from `print` / `println` goes to `System.out`. When the program finishes, the final line is:

```
Process exits with <return_value>.
```

Example

```
int main() {
    println("Hello, World!");
    return 0;
}
```

Output:

```
Hello, World!
Process exits with 0.
```

7.2 Error Output

Error	Exit Code	Output line
Assertion failure	33	`Process exits with 33.`
Null pointer	34	`Process exits with 34.`
Array out of bounds	34	`Process exits with 34.`
Type error	34	`Process exits with 34.`
Other runtime error	34	`Process exits with 34.`

Print all output (including the error line) to `System.out`. You may use `System.err` for debug output — it will not be checked.

Note: Do not throw any class which implements `Error` at any time. Use `Exception` instead. Throwing an `Error` will lead to a Wrong Answer in our automatic testing.

Task 8: Submission

Submit your source files as a zip to our OJ system. OJ will use the maven files to build the interpreter. OJ then tests the correctness of the output. We do not consider the efficiency of the interpreter in this lab.

The `.zip` should follow the same structure as previous labs:

```
submission.zip
└── outer_folder/
    ├── src/
    └── pom.xml
```